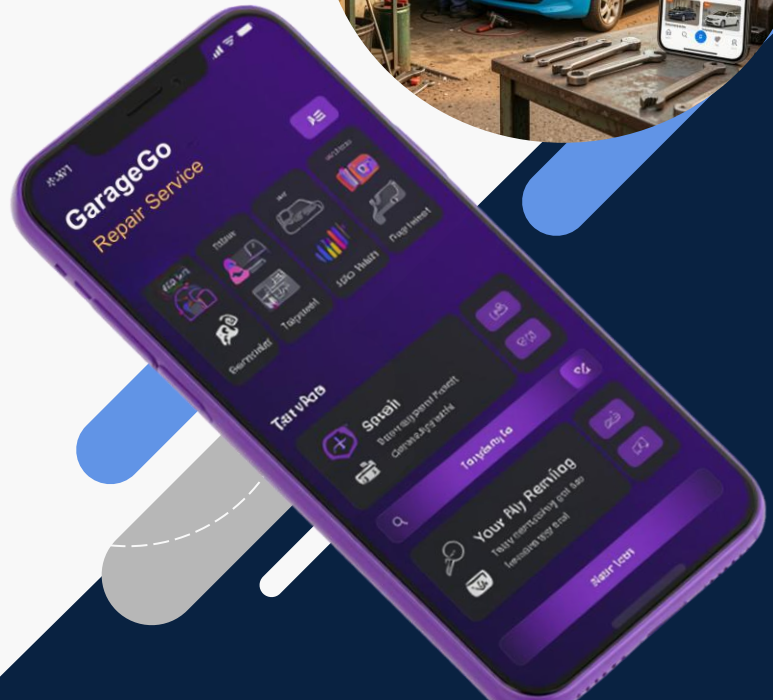
GarageGo

Smart Garage & Emergency Vehicle Repair Platform

Connecting Sri Lankan Vehicle Owners with Trusted Garages — Anytime, Anywhere



Team - Digital Architects
1.SE/2022/053 - Kasturi Pushpanathan
2.SE/2022/048 - Sriharan Krishnapiriyan
3.SE/2022/056 - Kajaluxmy Kanesarasa
4.SE/2022/031 - Vasanthan Vanushan

Software Design Specification style presentation

SENG 31242 – System Design Project

University of Kelaniya

System Overview

Core users, platform channels, and overall business objective

Customer

Searches trusted garages, books services, triggers emergency SOS, and tracks assigned mechanics.

Garage Owner

Publishes garage details, manages bookings, accepts emergency requests, and supervises service operations.

Mechanic

Receives assigned jobs, updates live location, confirms arrival, and completes repairs.

Administrator

Approves garages, monitors platform usage, resolves complaints, and enforces service quality.

Customer Mobile App React Native

Garage Owner web App React Native

Mechanic Mobile App React Native

Admin Dashboard React

Shared Backend (Node.js)

Goal: "Provide vehicle owners with trusted garage discovery, scheduled service booking, emergency SOS assistance, and real-time mechanic tracking."

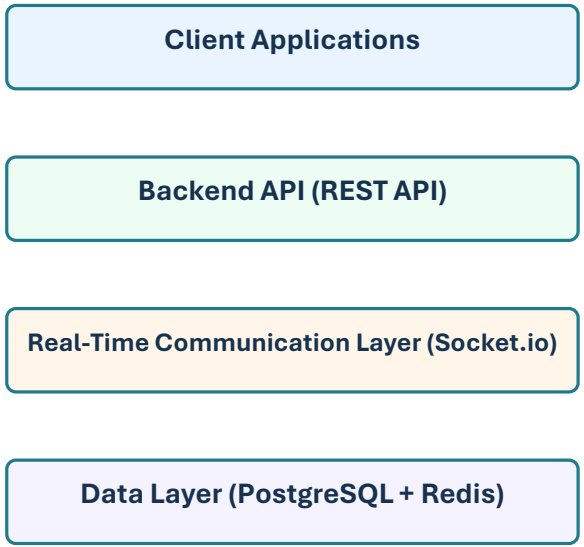
Architecture Pattern

Hybrid architecture balancing modularity for business operations and responsiveness for emergency dispatch

Layered Architecture

- Separates presentation, business logic, communication, and persistence responsibilities.
- Improves maintainability and supports easier testing for feature modules.
- Provides clear interfaces between mobile apps, APIs, and storage services.

Architecture Layers



Event-Driven Architecture

- Emergency broadcasts and mechanic tracking depend on asynchronous event delivery.
- Redis-backed signaling can reduce notification latency during high demand.
- Supports scalability for concurrent SOS incidents and booking updates.

Quality Attributes

- Scalability
- Maintainability
- Reliability

Technology Stack

Selected technologies and their contribution to the GarageGo solution

React Native (Expo)

Purpose: Builds customer, owner, and mechanic mobile apps from one codebase.
Why selected: Accelerates cross-platform delivery and simplifies maintenance.

React.js

Purpose: Implements the administrative web dashboard.
Why selected: Supports modular UI development and rich monitoring features.

Node.js + Express

Purpose: Provides REST endpoints and core service orchestration.
Why selected: Handles asynchronous requests efficiently and suits JavaScript full-stack development.

PostgreSQL

Purpose: Stores transactional data such as bookings, vehicles, and reviews.
Why selected: Offers ACID compliance, relational integrity, and mature indexing.

Redis

Purpose: Caches frequent queries and supports fast temporary state handling.
Why selected: Reduces latency for active dispatch and session-related workloads.

Socket.io

Purpose: Enables live event delivery and location updates.
Why selected: Simple real-time bi-directional communication for SOS and tracking.

Google Maps API

Purpose: Supplies maps, routing, geocoding, and ETA calculations.
Why selected: Reliable location services are essential for garage discovery and emergency response.

Firebase Cloud Messaging

Purpose: Delivers push notifications to mobile users.
Why selected: Supports timely alerts for bookings, approvals, and emergencies.

JWT Authentication

Purpose: Secures stateless user authentication across clients.
Why selected: Fits distributed APIs and role-aware session handling.

Cloudinary / AWS S3

Purpose: Stores vehicle images, documents, and garage media assets.
Why selected: Provides scalable, durable cloud-based asset storage.

Component Diagram

High-level service decomposition and interaction channels across the platform

Customer Mobile App

Garage Owner App

Mechanic App

Admin Dashboard

GarageGo REST API

Authentication Controller

Booking Controller

Emergency Controller

Garage Controller

Notification Controller

PostgreSQL Database

Redis Cache

Cloud Storage

External Services

- Google Maps API
- Twilio
- Firebase

Design notes: Client applications invoke the REST API for secure transactional operations, Socket-based notifications coordinate live emergency events, and external services extend mapping, messaging, and push delivery capabilities.

Deployment Diagram

Runtime environment for mobile clients, administrative access, and supporting cloud services

User Devices

- Customer Phone
- Garage Owner web
- Mechanic Phone
- Admin Workstation

Cloud Infrastructure

- API Server
- Socket.io Server
- Cloud Storage
- PostgreSQL Database
- Redis Server

Communication

- HTTPS secures API requests between client apps and backend services.
- WSS supports encrypted real-time event streaming for emergency dispatch and mechanic tracking.

Data Model

Core domain entities and relational structure supporting transactional and emergency workflows

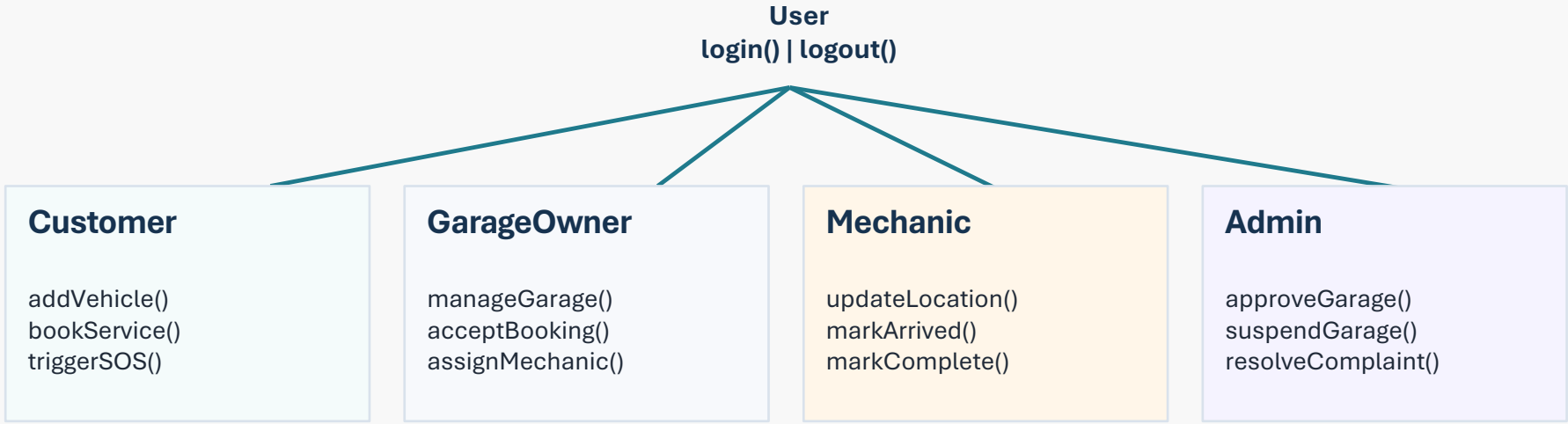
Entity	Key Relationship / Responsibility
Customer	Owns one or many vehicles, creates bookings, emergency requests, reviews, and notifications.
Vehicle	Belongs to a customer and maintains service history across bookings and emergency interventions.
Garage	Managed by a garage owner and linked to mechanics, bookings, reviews, and emergency eligibility.
GarageOwner	Supervises one or many garages and handles operational decisions for services and assignments.
Mechanic	Assigned to bookings or emergency requests and publishes status plus live location updates.
Booking	Connects customer, vehicle, garage, and mechanic within a scheduled service transaction.
EmergencyRequest	Captures SOS incidents, location, dispatch state, assigned garage, and assigned mechanic.
Review	Represents post-service customer feedback tied to a completed garage interaction.
Notification	Stores system-generated alerts for booking events, approvals, and emergency progress.
Admin	Maintains governance actions such as garage approval, suspension, and dispute resolution.

Design Notes

- ACID compliance in PostgreSQL protects booking confirmation and payment-related consistency.
- Relational links support traceability among customers, vehicles, garages, mechanics, and service outcomes.
- Vehicle-centered service history enables repeat maintenance analysis and improved recommendations.
- Emergency request lifecycle records creation, broadcast, acceptance, dispatch, completion, and closure states.

Class Design

Inheritance and domain classes organizing responsibilities within the business model



Related Domain Classes

Vehicle | Booking | EmergencyRequest | Garage | Review | Notification

The user hierarchy centralizes authentication behavior, while specialized subclasses encapsulate role-specific operations and collaborate with independent domain entities for service execution.

Key Workflow – Emergency SOS Dispatch

Sequence of control for time-critical roadside assistance and live customer visibility

Customer App

Emergency Controller

Google Maps API

Socket.io Server

Eligible Garages

First Garage Accepts

Mechanic Assigned

Live GPS Tracking

Customer Receives ETA

Workflow Design Controls

- First-Accept-Wins assignment prevents multiple garages from claiming the same incident.
- Database locking with SELECT FOR UPDATE protects dispatch consistency during concurrent acceptance attempts.
- Real-time tracking updates every 5 seconds to keep the customer informed of mechanic progress.
- Fallback logic expands the search radius when nearby garages fail to respond within the initial dispatch window.

This workflow prioritizes low-latency event propagation, transactional integrity, and customer assurance during breakdown scenarios.

Security & Quality Attributes

Security mechanisms and non-functional requirements guiding platform robustness

Authentication

- JWT
- Refresh Tokens

Access Control

- Role-Based Access Control

Data Protection

- HTTPS/TLS
- BCrypt Password Hashing

Input Validation

- Request Validation
- Sanitization

Reliability

- Redis Caching
- Automated Backups

Scalability

- Horizontal Scaling
- Socket.io Real-Time Communication

Performance

- Emergency Broadcast < 2 seconds
- API Response < 2 seconds

Collectively, these controls aim to preserve confidentiality, ensure resilient service delivery, and maintain acceptable response times for both routine bookings and emergency incidents.